
Vehicle Market Analytics System

Data Processing & Dataflow Documentation

Milestone 3 — Dataflow Documentation & Dataset Preprocessing Summary

Prepared By	Muddassir Khan, Salman Khan
Instructor	Ali Hassan
Course	Introduction to Database Systems
Institution	Institute of Management Sciences, Peshawar
Date	May 2026

Dataset Summary			
497	18	12	3NF
Clean Records	Columns	DB Tables	Normal Form

Contents

1	Introduction	3
2	Dataset Preprocessing Summary	3
2.1	Duplicate Removal	3
2.2	Missing Value Handling	3
2.3	Data Type Correction	4
2.4	Standardization	4
2.5	Column Reduction	4
2.6	Final Dataset Statistics	5
3	Dataflow Documentation	6
3.1	Data Input Layer	6
3.1.1	CSV Dataset Import	6
3.1.2	Live User Input	6
3.1.3	Image URL Handling	6
3.2	Database Layer (MySQL)	6
3.2.1	Table Population Order	7
3.2.2	Key Relationships	7
3.2.3	ETL Row Resolution Chain	7
3.3	Backend Layer (Flask)	7
3.3.1	API Endpoints	8
3.3.2	Validation and Query Patterns	8
3.4	Frontend Layer	8
3.5	Output Layer	8
3.6	End-to-End Workflow Summary	9

1. Introduction

The **Vehicle Market Analytics System** is a full-stack, data-driven platform that aggregates and presents used vehicle listings from **PakWheels**, Pakistan’s leading automotive market-place. It allows users to explore market trends, compare prices, and filter listings by make, model, year, city, fuel type, and transmission.

The system consists of four integrated components:

- A preprocessed CSV dataset (`pakwheels_cleaned.csv`) — 497 clean records.
- A normalised MySQL relational database (`pakwheels_db`) — 3NF, 12 tables.
- A Python Flask RESTful backend — validated API endpoints for CRUD and analytics.
- A dynamic HTML/CSS/JavaScript frontend — interactive listings dashboard.

This document covers Milestone 3: the *dataset preprocessing pipeline* and the *complete dataflow* through each system layer, from raw input to user-facing output.

2. Dataset Preprocessing Summary

The raw dataset was scraped from PakWheels on **21 May 2026**, containing **500 records** across **23 columns**. After preprocessing, the output file contained **497 records** across **18 columns**, with zero null values and full MySQL compatibility.

2.1 Duplicate Removal

Three duplicate rows were identified and removed using full-row comparison and unique ID-based deduplication.

2.2 Missing Value Handling

Table 1: *Missing value treatment by field*

Field	Strategy	Detail
make, model, price, year	Remove row	Critical fields; rows with missing values dropped.
city	Default value	Replaced with 'Unknown'.
color	Default value	Replaced with 'Unspecified'.
fuel_type	Default value	Defaulted to 'Petrol'.
transmission	Default value	Defaulted to 'Automatic'.
price_note	Default value	Defaulted to 'Direct'.
image, detail_url	Null	Invalid or missing URLs set to NULL.

2.3 Data Type Correction

Table 2: Data type corrections applied during preprocessing

Field	Original Type	Corrected Type
price	Object / String	Integer (Int64)
year	Float	Integer (Int64)
mileage_km	Float	Integer (Int64)
engine_cc	Float	Integer (Int64)
id	Float	Integer (Int64)
scraped_at	String	Datetime (YYYY-MM-DD HH:MM:SS)
Text fields	Mixed case	Stripped & title-cased strings

2.4 Standardization

- All text fields stripped of leading/trailing whitespace; normalized to Title Case.
- Price currency standardized to 'PKR' throughout.
- Year values validated within range **1970–2026**; out-of-range records removed.
- Price and mileage validated as positive integers; zero or negative records removed.

2.5 Column Reduction

Five non-essential columns were dropped: `description`, `thumbnail`, `url_list_page`, `from_detail`, `source`. The retained 18 columns cover all fields required for database population and analytics.

2.6 Final Dataset Statistics

Table 3: Top vehicle makes by listing count

#	Make	Listings
1	Toyota	157
2	Honda	127
3	Suzuki	91
4	Kia	24
5	Daihatsu	20
6	Nissan	14
7	Hyundai	14
8	Haval	6
9	Mercedes Benz	6
10	Mitsubishi	6

Table 4: Listings by city (top 10)

#	City	Listings
1	Lahore	128
2	Islamabad	102
3	Karachi	93
4	Rawalpindi	43
5	Faisalabad	25
6	Multan	18
7	Peshawar	9
8	Sargodha	8
9	Sialkot	7
10	Bahawalpur	7

Table 5: Fuel type, transmission, and price summary

Fuel Type		Transmission	
Petrol	422	Automatic	352
Hybrid	44	Manual	145
Diesel	23		
PHEV/Other	8		

Table 6: Price range summary (PKR)

Metric	Value
Minimum Price	260,000
Maximum Price	55,000,000
Average Price	5,180,529
Year Range	1986 – 2026

3. Dataflow Documentation

This section describes the complete dataflow through each layer of the system, from raw data input to user-facing output.

3.1 Data Input Layer

3.1.1 CSV Dataset Import

The cleaned CSV file is imported using a Python ETL script that reads each row via `pandas`, validates field types, and inserts records into the MySQL database in strict referential order — parent tables before child tables.

3.1.2 Live User Input

The system also accepts live input through the frontend and admin panel: search queries, filter selections (make, city, year, fuel type, price range), and admin-level CRUD operations via secured API endpoints.

3.1.3 Image URL Handling

Image URLs are validated during preprocessing to confirm they start with `https://` and point to PakWheels CDN paths. Invalid or missing URLs are stored as `NULL`; the frontend substitutes a placeholder image via an `onerror` handler.

3.2 Database Layer (MySQL)

3.2.1 Table Population Order

Table 7: ETL table insertion order to satisfy referential integrity

#	Table	Reason
1	locations	No FK dependencies; referenced by sellers and vehicles.
2	categories	Reference table for vehicle type classification.
3	users	Parent of sellers and admins.
4	admins	Extends users; requires users to exist first.
5	sellers	References users and locations.
6	vehicles	Core table; references sellers, locations, categories.
7	vehicle_images	Child of vehicles; stores additional image URLs.
8	favorites	Junction table: users × vehicles.
9	comparisons	References users and up to three vehicles.
10	inquiries	References users, vehicles, and sellers.
11	reviews	References reviewer (user) and seller.
12	search_logs	Analytics log; no blocking FK dependencies.

3.2.2 Key Relationships

- `vehicles.seller_id → sellers.seller_id` (Many-to-One)
- `vehicles.location_id → locations.location_id` (Many-to-One)
- `vehicles.category_id → categories.category_id` (Many-to-One)
- `vehicle_images.vehicle_id → vehicles.vehicle_id` (Many-to-One)
- `sellers.user_id → users.user_id` (One-to-One)
- `favorites(user_id, vehicle_id) → users, vehicles` (Junction)

3.2.3 ETL Row Resolution Chain

When each CSV row is processed, the ETL script:

1. Resolves or inserts the **location** and retrieves its primary key.
2. Resolves or inserts the **user** and **seller** records linked to that location.
3. Inserts the **vehicle** record referencing all resolved foreign keys.
4. Inserts the **image URL** into `vehicle_images` linked by `vehicle_id`.

3.3 Backend Layer (Flask)

3.3.1 API Endpoints

Table 8: Flask REST API endpoint reference

Endpoint	Description
GET /api/vehicles	Paginated listings with filters: make, city, year, fuel, price range, transmission.
GET /api/vehicles/<id>	Full details of a single vehicle by ID.
POST /api/vehicles	Insert new vehicle after field validation and FK resolution.
PUT /api/vehicles/<id>	Update existing vehicle record with type-checked fields.
DELETE /api/vehicles/<id>	Remove a vehicle record by ID.
GET /api/analytics	Aggregated stats: avg price by make, count by city, fuel/year breakdown.
GET /api/makes	All unique makes for dropdown population.
GET /api/cities	All unique cities for filter and search.

3.3.2 Validation and Query Patterns

All POST/PUT requests are validated for required fields before any database write. Numeric fields are type-checked and range-validated. Parameterized queries via SQLAlchemy prevent SQL injection. Multi-table inserts are wrapped in transaction blocks for atomicity. Structured JSON error responses with HTTP 400/404/500 codes are returned on failure.

3.4 Frontend Layer

Data is fetched asynchronously via the `fetch()` API with JSON parsing:

- **On load:** GET /api/analytics populates summary cards and Chart.js charts.
- **Listings page:** GET /api/vehicles with active query parameters for filters.
- **Detail view:** GET /api/vehicles/<id> triggered on card click.
- **Dropdowns:** populated via GET /api/makes and GET /api/cities.

Filter inputs are serialized as URL query parameters. Results update dynamically without page reload. Pagination and sort options (price, year) trigger re-fetches with LIMIT/OFFSET and ORDER BY parameters.

3.5 Output Layer

- **Dashboard cards:** total listings, average price, most-listed make, top city.
- **Charts (Chart.js):** listings by make (bar), average price by city (bar), fuel distribution (pie), transmission split (pie), listings by year (line).
- **Vehicle cards:** image, title, price (PKR formatted), year, city, mileage, transmission.
- **Admin tables:** sortable summaries by city, make, or price band via analytics endpoints.

3.6 End-to-End Workflow Summary

Table 9: Complete system data pipeline

#	Stage	Detail
1	Raw Dataset	500 records, 23 columns scraped from PakWheels (21 May 2026).
2	Preprocessing	Deduplication, type correction, null handling, standardization via pandas.
3	Clean CSV	497 records, 18 columns, UTF-8, MySQL-ready.
4	MySQL Import	ETL script populates 12 normalised tables in dependency order.
5	Flask Backend	REST API serves CRUD operations and analytics via parameterized JSON queries.
6	Frontend	HTML/JS dashboard renders listings, charts, and filters dynamically.
7	User Output	Vehicle cards, analytics dashboard, search results, and admin reports.